RAG

What is RAG?

- Combines retrieval (external knowledge) + generation (LLMs)
- Addresses LLM limitations: static knowledge, hallucinations
- Retrieval + Generation = RAG

RAG vs Fine-tuning

- Prompt Engineering involves
  - designing and optimizing the input prompts to elicit the best possible responses.
  - This technique focuses on crafting specific queries or instructions that guide the model's output.
  - The effectiveness of prompt engineering can significantly influence the quality of the generated text, making it a crucial skill for AI practitioners.

Why RAG Matters?

1. Up-to-date information
2. Source attribution (trust)
3. Cost-effective vs. fine-tuning

RAG Workflow

1. Query
2. Retrieve
3. Augment
4. Generation

Retriever Module

- Sparse (keyword)
- Dense (semantic)
- Tools
  - Vecto DB (FAISS, Chroma, Pinecone)

Generator Module

- LLMs
- *Blend* retrieved context + pre-trained knowledge

Vector DB

- Store embeddings for fast similarity search
- Indexing
- Retriever

Embedding Models

- OpenAI's Ada-002, Jina AI
- Applying cosine similarity, Euclidian distance

Advanced Techiniques

- Domain specific embeddings
- Query expansion (pseudo-relevance)

Chunking Strategies

- Parent-document retrieval
- Multi-vector indexing

Advance RAG architectures: Fine-tuning RAG

- Retrieval-Augment Fine-tuning

Advance RAG architectures: Context RAG

- Smart context retrieval system that enhances both tool retrieval and plan generation
- Uses lightweight model with numerical, categorical, and habitual usage signals
- Achieves 3.5x improvement in context retrieval and 1.5x in tool retrieval tasks
- Reduces hallucination by augmenting context at plan generation

Advance RAG architectures: GraphRAG

- Organizes textual information into an indexing graph
- Models complex relationships between data points for improved precision
- Captures inherent dependencies across text database
- Improves answer precision by up to 35% compared to vector-only methods
- Enables deeper and more contextual retrieval through entity relationship modeling

Advance RAG architectures: LightRAG

- Incorporates graph structures into text indexing and retrieval
- Employs dual-level retrieval system for comprehensive information discovery
- Integrates graph structures with vector representations for efficient entity relationship retrieval
- Features incremental update algorithm for timely integration of new data

Advance RAG architectures: PathRAG

- Retrieves key relational paths from indexing graphs
- Converts paths into textual form for LLM prompting
- Reduces redundancy with flow-based pruning algorithms
- Guides LLMs to generate more logical and coherent responses48
- Consistently outperforms other RAG approaches across multiple evaluation dimensions

Advance RAG architectures: Sufficient Context Evaluation

- New framework for evaluating RAG system performance
- Classifies whether retrieved context is sufficient to answer queries
- Enables selective generation to reduce hallucinations
- Guides models to abstain when context is insufficient5
- Improves accuracy by 2-10% in practical applications

Real World Applications

- Healthcare diagnositc support systems
- Legal research and case analysis
- E-commerce assistants (e.g. Shopify Sidekick)

Evaluation Metrics

- Retrieval: Precision@K, Recall@K, NDCG
- Generation: Faithfulness, hallucination rate, contextual relevance
- End-to-End: Human evaluation, task completion rate

Future Trends

- Self-retrieving language models
- Multimodal RAG (images/audio/video)
- Hybrid symbolic-neural architectures

"RAG is the bridge between LLMs and real-world dynamism."

Summary

- RAG architectures continue to evolve, offering increasingly sophisticated ways to ground LLM responses in accurate, contextual information.